absolute prediction error but instead to the rate of change of prediction error. When the agent transitions to an unpredictable state, it will experience a transient surge of prediction error, but then it goes away. Similarly, if the agent encounters a noisy TV, at first it is highly unpredictable and therefore has a high prediction error, but the high prediction error is maintained, so the rate of change is zero.

This formulation is better, but it still has some potential issues. Imagine that an agent is outside and sees a tree with leaves blowing in the wind. The leaves are blowing around randomly, so this is a high prediction error. The wind stops blowing, and the prediction error goes down, since the leaves are not moving anymore. Then the wind starts blowing again, and prediction error goes up. In this case, even if we're using a prediction error rate, the rate will be fluctuating along with the wind. We need something more robust.

We want to use this prediction error idea, but we don't want it to be vulnerable to trivial randomness or unpredictability in the environment that doesn't matter. How do we add in the "doesn't matter" constraint to the prediction error module? Well, when we say that something doesn't matter, we mean that it does not affect us or is perhaps uncontrollable. If leaves are randomly blowing in the wind, the agent's actions don't affect the leaves, and the leaves don't affect the actions of the agent. It turns out we can implement this idea as a separate module, in addition to the state prediction module—that's the subject of this chapter. This chapter is based on elucidating and implementing the idea from a paper by Deepak Pathak et al. titled "Curiosity-driven Exploration by Self-supervised Prediction" (2017), which successfully resolves the issues we've been discussing.

We will follow this paper pretty closely because it was one of the biggest contributions to solving the sparse reward problem, and this paper led to a flurry of related research. It also turns out to describe one of the easiest algorithms to implement, among the many others in this area. In addition, one of the goals of this book is to not only teach you the foundational knowledge and skills of reinforcement learning, but to give you a solid-enough mathematics background to be able to read and understand reinforcement learning papers and implement them on your own. Of course,